

# Complete Step-by-Step Setup for Redux Toolkit and RTK Query in E-commerce Project

## Suggested Folder Structure:

To keep everything organized, here is a suggested folder structure for your project:

```
/src
  /features
    /auth
      - authSlice.js // Auth reducer (JWT handling)
    /products
      - productsSlice.js // Product reducer (Product state management)
  /services
    - apiConfig.js // API configuration (base URL, headers)
    - authApi.js // Auth-related queries and mutations
    - productsApi.js // Product-related queries and mutations
  /reducers
    - rootReducer.js // Combine all reducers here
  /hooks
    - index.js // Custom hooks like useAppDispatch, useAppSelector
  /store
    - store.js // Store configuration
```

## Step 1: Install Dependencies

First, we need to install the required packages:

```
npm install @reduxjs/toolkit react-redux
```

- **@reduxjs/toolkit**: Provides the tools for state management.
- **react-redux**: Provides the bindings to use Redux with React.

## Step 2: Set Up Redux Store with Middleware

Configure the Redux store to use **RTK Query** middleware and the **root reducer**.

**store.js:**

```
import { configureStore } from "@reduxjs/toolkit";
import rootReducer from "@reducers"; // Root reducer to combine all slices
import apiConfig from "@services/apiConfig"; // API config for RTK Query
```

```
const store = configureStore({
  reducer: rootReducer, // Use rootReducer to combine slices
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(apiConfig.middleware), // Add RTK Query middleware
});

export default store;
```

### Step 3: Create API Configuration (Setting Base URL and Headers)

In this step, we will set up the **base URL** and handle **headers conditionally** (e.g., adding the **JWT token** for protected routes).

**apiConfig.js:**

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

// Base URL from environment variables
const baseUrl = `${import.meta.env.VITE_API_BASE_URL}`;

// Base query configuration for RTK Query
const baseQuery = fetchBaseQuery({
  baseUrl, // Use the base URL from environment variables
  prepareHeaders: (headers, { getState, endpoint }) => {
    const token = localStorage.getItem("token"); // Get token from localStorage

    // Add token to the header for most requests, but exclude certain endpoints
    if (token && endpoint !== "getProducts" && endpoint !== "getProduct") {
      headers.set("authorization", `Bearer ${token}`);
    }

    return headers;
  },
});

// API configuration using `createApi`
const apiConfig = createApi({
  reducerPath: "api",
  baseQuery,
  endpoints: () => ({}), // Empty endpoints for now
});

export default apiConfig;
```

**Explanation:**

- **baseUrl:** Set from environment variables (you can configure this in a `.env` file).
- **prepareHeaders:** This function adds the **JWT token** to the request headers unless the request is for public endpoints (e.g., `getProducts`).

## Step 4: Create the Slices (Auth and Product Slices)

We'll now create two slices: one for **auth** (to handle JWT token) and one for **products**.

### Auth Slice (**features/auth/authSlice.js**):

```
import { createSlice } from "@reduxjs/toolkit";

// Initial state for authentication
const initialState = {
  token: localStorage.getItem("token") || null, // Get token from localStorage
};

const authSlice = createSlice({
  name: "auth", // This is the name used to reference the auth state
  initialState,
  reducers: {
    setToken: (state, action) => {
      state.token = action.payload;
      localStorage.setItem("token", action.payload); // Save token to localStorage
    },
    logout: (state) => {
      state.token = null;
      localStorage.removeItem("token"); // Remove token from localStorage
    },
  },
});

// Action creators
export const { setToken, logout } = authSlice.actions;

// Selector to access auth state
export const getAuthState = (state) => state.auth;

export default authSlice.reducer;
```

### Product Slice (**features/products/productsSlice.js**):

```
import { createSlice } from "@reduxjs/toolkit";

// Initial state for products
const initialState = {
  products: [],
  selectedProduct: null
};

const productSlice = createSlice({
  name: "products", // This name will be used to reference the products state
  initialState,
  reducers: {
    setProducts: (state, action) => {
```

```

    state.products = action.payload;
  },
  setSelectedProduct: (state, action) => {
    state.selectedProduct = action.payload;
  },
},
});

// Action creators
export const { setProducts, setSelectedProduct } = productSlice.actions;

// Selector to access products state
export const getProductsState = (state) => state.products;

export default productSlice.reducer;

```

## Step 5: Combine Reducers (Root Reducer)

Combine all slices (auth and product) and the **API configuration** (from RTK Query).

**rootReducer.js:**

```

import { combineReducers } from "@reduxjs/toolkit"; // Import combineReducers
import apiConfig from "@services/apiConfig"; // API config for RTK Query
import authReducer from "@features/auth/authSlice"; // Auth slice for managing JWT token
import productReducer from "@features/products/productsSlice"; // Product slice for managing product state

const rootReducer = combineReducers({
  auth: authReducer, // Auth state for JWT token
  products: productReducer, // Products related state
  [apiConfig.reducerPath]: apiConfig.reducer, // Add the apiConfig reducer for RTK Query
});

export default rootReducer;

```

## Step 6: Create API Endpoints and Use in Components

We will now define the **API endpoints** for managing products, authentication, etc.

**API Endpoints for Products (services/productsApi.js):**

```

import apiConfig from "@services/apiConfig";

const addTagTypes = ["product"];
const ProductApi = apiConfig.enhanceEndpoints({
  addTagTypes,
}).injectEndpoints({

```

```

endpoints: (builder) => ({
  getProducts: builder.query({
    query: () => "/products", // Fetch all products
    providesTags: (result) =>
      result ? [{ type: "product", id: "LIST" }] : [],
  }),

  getProduct: builder.query({
    query: (id) => `/products/${id}`, // Fetch specific product by ID
  }),

  postProduct: builder.mutation({
    query: (product) => ({
      url: "/admin/products",
      method: "POST",
      body: product,
    }),
    invalidatesTags: ["product"],
  }),

  updateProduct: builder.mutation({
    query: ({ id, product }) => ({
      url: `/admin/products/${id}`,
      method: "PATCH",
      body: product,
    }),
    invalidatesTags: ["product"],
  }),
}),
});

export const {
  useGetProductsQuery,
  useGetProductQuery,
  usePostProductMutation,
  useUpdateProductMutation,
} = ProductApi;

```

#### API Endpoints for Auth (**services/authApi.js**):

```

import apiConfig from "@services/apiConfig";

const AuthApi = apiConfig.enhanceEndpoints({
  addTagTypes: ["auth"],
}).injectEndpoints({
  endpoints: (builder) => ({
    login: builder.mutation({
      query: (credentials) => ({
        url: "/auth/login",
        method: "POST",
        body: credentials,
      }),
    }),
  }),
});

```

```

    })),
    logout: builder.mutation({
      query: () => ({
        url: "/auth/logout",
        method: "POST",
      }),
    }),
  }),
});

export const { useLoginMutation, useLogoutMutation } = AuthApi;

```

## Step 7: Using Queries and Mutations in Components

### Fetching Products with `useGetProductsQuery`

```

import { useGetProductsQuery } from "@services/productsApi"; // Import query hook
for fetching products

function ProductList() {
  const { data: products, isLoading, error } = useGetProductsQuery();

  if (isLoading) return <div>Loading products...</div>;
  if (error) return <div>Error: {error.message}</div>;

  return (
    <div>
      {products.map((product) => (
        <div key={product.id}>
          <h3>{product.name}</h3>
          <p>{product.description}</p>
        </div>
      ))}
    </div>
  );
}

```

### Login with `useLoginMutation`

```

import { useLoginMutation } from "@services/authApi"; // Import auth mutation
import { useAppDispatch } from "@hooks"; // Dispatch hook
import { setToken } from "@features/auth/authSlice"; // Set token action

function Login() {
  const [login, { isLoading, error }] = useLoginMutation();
  const dispatch = useAppDispatch();

  const handleLogin = async (credentials) => {
    try {
      const response = await login(credentials).unwrap();
      dispatch(setToken(response.token)); // Save token in Redux
    }
  };
}

```

```

    localStorage.setItem("token", response.token); // Save token in localStorage
  } catch (err) {
    console.error(err);
  }
};

return (
  <div>
    <button onClick={() => handleLogin({ email: "user@example.com", password:
"password" })}>
      Login
    </button>
    {isLoading && <div>Logging in...</div>}
    {error && <div>Error: {error.message}</div>}
  </div>
);
}

```

## Step 8: Using `useAppSelector` to Access Redux State

To access the state managed by Redux, we use the `useAppSelector` hook.

### Example: Accessing Auth State

```

import { useAppSelector } from "@hooks"; // Custom selector hook
import { getAuthState } from "@features/auth/authSlice"; // Import the selector
function

function UserProfile() {
  const { token } = useAppSelector(getAuthState); // Access the auth state using the
  selector

  return <div>{token ? "User is logged in" : "Please log in"}</div>;
}

```

### Example: Accessing Products State

```

import { useAppSelector } from "@hooks"; // Custom selector hook
import { getProductsState } from "@features/products/productsSlice"; // Import the
selector function

function ProductList() {
  const { products } = useAppSelector(getProductsState); // Access the products
  state using the selector

  return (
    <div>
      {products.map((product) => (
        <div key={product.id}>
          <h3>{product.name}</h3>
          <p>{product.description}</p>

```

```

        </div>
      )}}
    </div>
  );
}

```

## Final Conclusion

This complete setup allows you to manage **auth state** (with JWT tokens), **products state**, and make API calls with **RTK Query** for fetching, adding, and updating products. Additionally, we've shown how to **use queries** and **mutations** in React components, as well as how to **use useAppSelector** to access Redux state.

### Provider

```

import React from "react";
import { Provider } from "react-redux";
import store from "@redux-state-management/store/store";

export default function ReduxProvider({
  children,
}): {
  children: React.ReactNode;
}) {
  return <Provider store={store}>{children}</Provider>;
}

```

### hooks/[index.js](#)

```

import { TypedUseSelectorHook, useDispatch, useSelector } from "react-redux";
import type {
  RootState,
  AppDispatch,
} from "@redux-state-management/store/store";

// Use throughout your app instead of plain `useDispatch` and `useSelector`
export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;

```